LABORATORUL 5

Reguli și prima partidă completă

Adăugăm hrană, creștere, scor, final și Restart în Snake.

- Observăm regulile în model și rezultatul în interfață.
- La final apare un dialog cu scorul, iar mâncarea și finalul au sunet.

2 Legătura cu cursul

Curs 2: interfață și lifecycle pentru afișarea rezultatului și oprirea actualizărilor.

- Curs 5, p. 5–13: AOSP și construirea sistemului; comparăm cu build-ul aplicației.
- Separarea regulilor pregătește Cursul 8. Breakpoint-ul pregătește Cursul 9.

Game Over apare într-un dialog peste ecranul jocului; nu deschidem altă Activity.

3 Stările rundei

RUNNING	update() poate modifica partida
GAME_OVER	partida s-a terminat prin pierdere
WON	partida s-a terminat prin victorie

```
if (status != Status.RUNNING) return;
```

- Starea modelului și running din GameLoop au roluri diferite: prima descrie partida, a doua programarea pașilor.

La final oprim bucla. Restart resetează modelul și reia aceeași buclă.

4 Plasarea hranei

- Hrana ocupă o celulă liberă, fără șarpe sau obstacol.

```
List<Cell> free = collectFreeCells();
if (free.isEmpty()) {
    food = null;
    status = Status.WON;
    return;
}
food = free.get(random.nextInt(free.size()));
```

- *collectFreeCells() este metoda noastră care enumeră celulele libere. Alegem hrana la început și după consumare.*

5 Creșterea și scorul

Stabilim dacă destinația conține hrană înainte de mutare.

```
boolean eating = food != null
    && nextCol == food.col && nextRow == food.row;
snake.add(0, new Cell(nextCol, nextRow));
if (eating) { score++; spawnFood(); }
else { snake.remove(snake.size() - 1); }
```

- La creștere păstrăm coada. Altfel lungimea rămâne constantă.

Comparăm coordonatele celulelor, nu referințele obiectelor.

6 Coliziuni și final

Calculăm destinația, verificăm regulile, apoi modificăm lista.

- Perete, corp sau obstacol înseamnă GAME_OVER; tabla plină înseamnă WON.
- Dacă nu mâncăm, ultima celulă se eliberează și nu participă la verificarea corpului.

```
int limit = eating ? snake.size() : snake.size() - 1;
```

Schimbarea față de Lab 4: marginea devine perete; eliminăm trecerea cu modulo.

7 Game Over și Restart

La final: bucla se oprește, se aude un ton și apare dialogul cu scorul.

- La Restart: resetăm șarpele, direcția, scorul, hrana, obstacolele și starea rundei la valorile inițiale.

```
gameLoop.stop();
boardView.restartGame();
refreshGameUi();
gameLoop.start();
```

- *Reutilizăm aceeași buclă și păstrăm viteza aleasă cu Lent/Rapid.*

8 Responsabilitățile claselor

GameState: reguli, poziții, scor și final.

- BoardView: desenează starea; nu decide coliziuni.
- Activity: leagă butoanele, bucla, dialogul de final și sunetele.

GameState nu folosește clase Android, deci poate fi testat fără emulator.

9

O regulă în debugger

Breakpoint-ul suspendă execuția pentru inspectarea datelor disponibile în acel punct.

- Punem breakpoint înainte de verificarea limitei tablei.
- Inspectăm nextCol, nextRow, dimensiunile și status.
- Step Over execută următoarea instrucțiune; Resume continuă rularea.

Debuggerul completează logurile și testele.

10 Cazuri-limită controlate

Tabla plină	obținem WON fără blocare
Intrarea în coada care pleacă	mutare validă dacă nu creștem
Restart repetat	nu apar două bucle
A doua rundă	scorul și starea pornesc corect
Hrana	nu apare într-o celulă ocupată

- Folosim stări pregătite și rezultate așteptate, nu doar joc aleatoriu.

11

Build-ul aplicației și AOSP

Build-ul aplicației: Gradle și Android Gradle Plugin compilează codul, procesează resursele și produc un APK.

- Build-ul AOSP: construiește componente și imagini ale platformei pentru o țintă aleasă.

```
./gradlew assembleDebug
```

- Pe Windows folosim gradlew.bat. Nu recompilăm Android când construim Snake.

Durata depinde de proiect, configurație și cache.